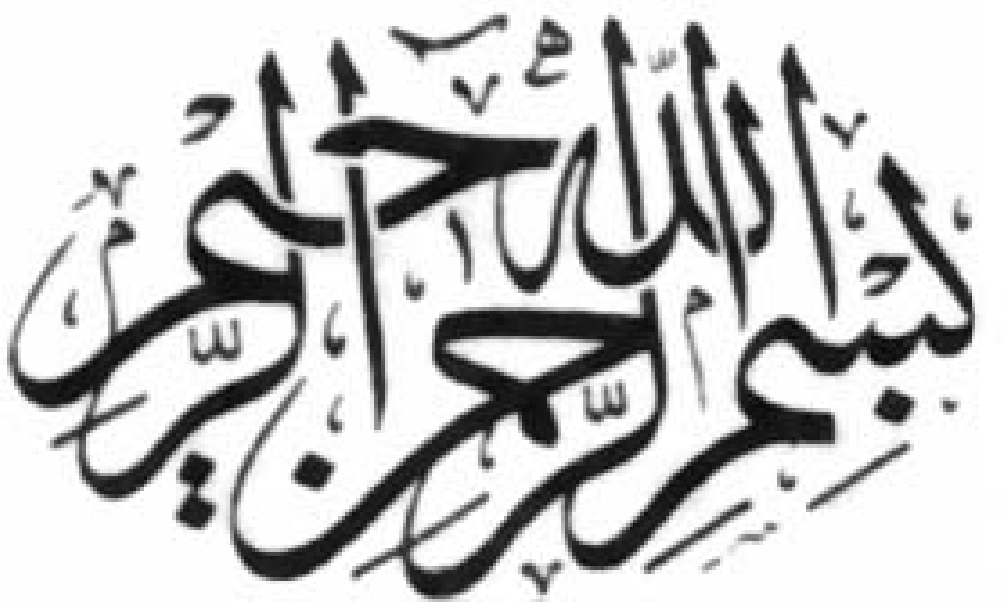

بسم الله الرحمن الرحيم

Micro-coded state machines

Micro-coded state machines

- In many applications the algorithms are so complex that a hardwired FSM design is not feasible.
- Several algorithms need to be implemented on the same data-path or the designer wants to keep the flexibility of modifying the controller without repeating the entire design cycle

Micro-coded state machines

- The combinational logic is replaced by a sequence of control signals that are stored in program memory (PM)

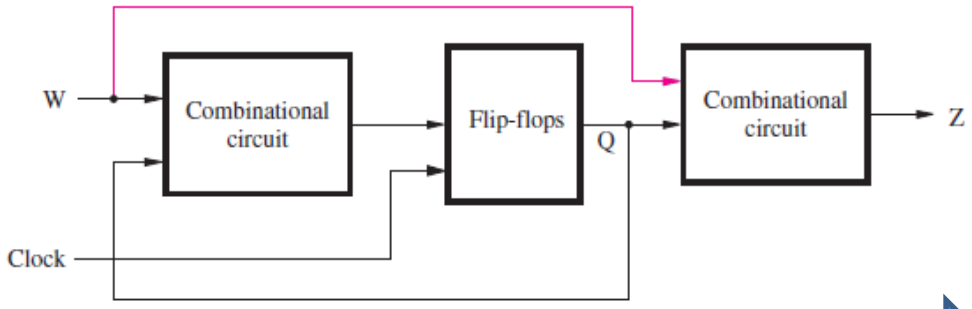
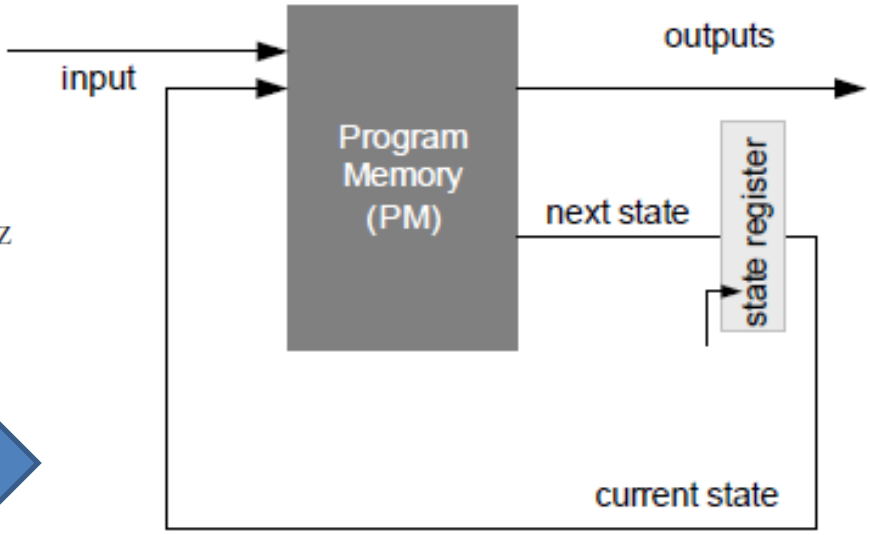
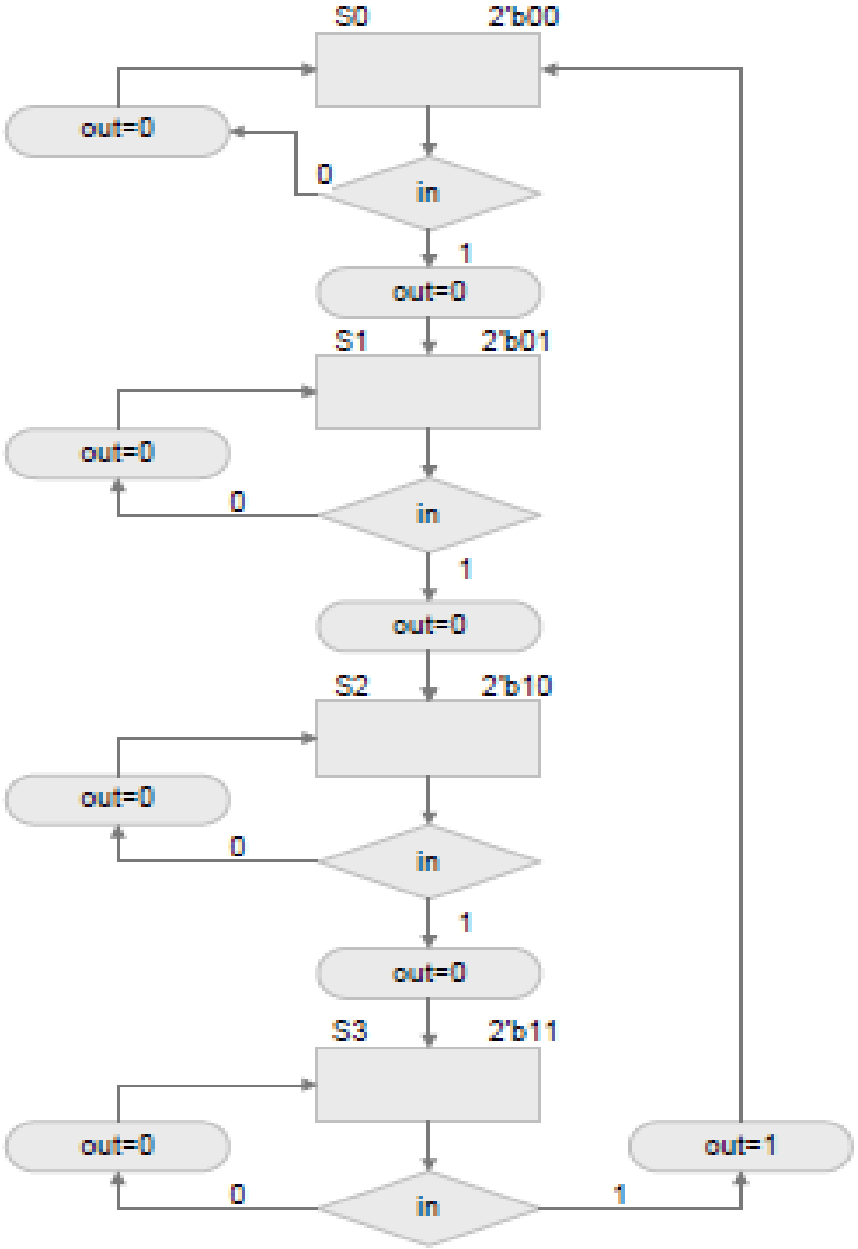
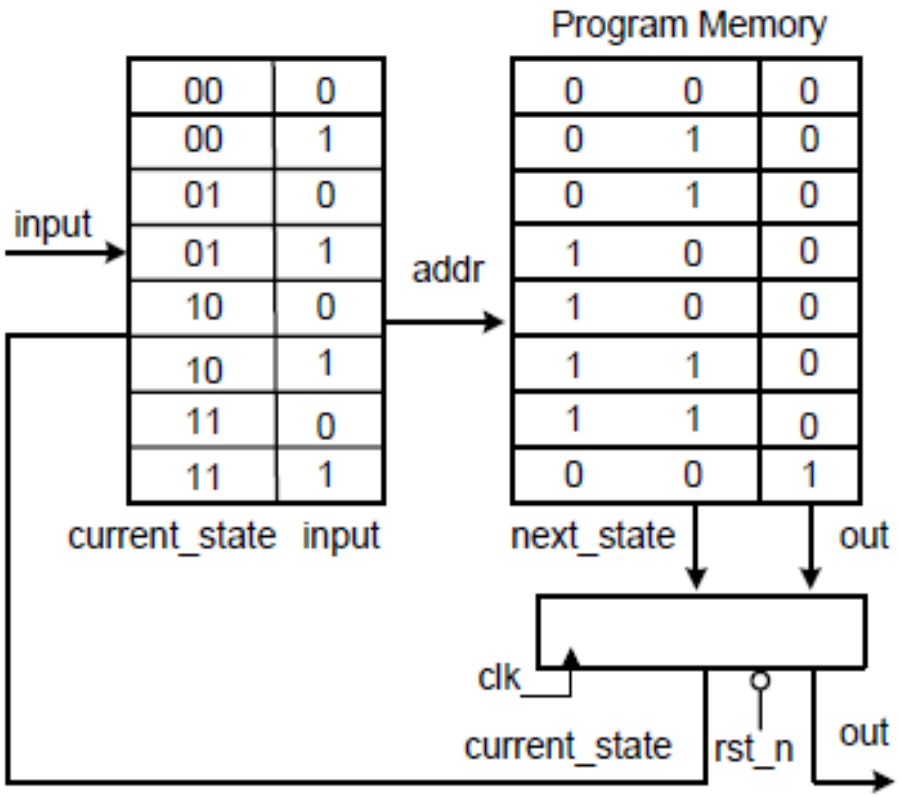


Figure 8.1 The general form of a sequential circuit.

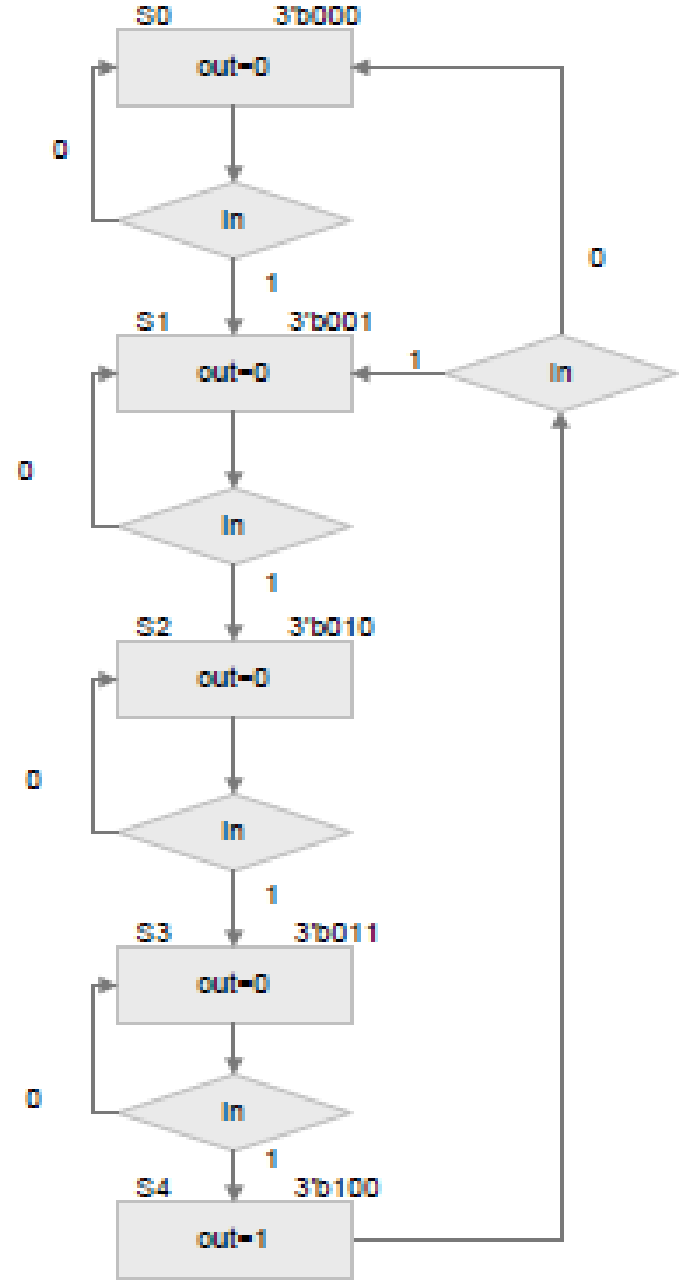


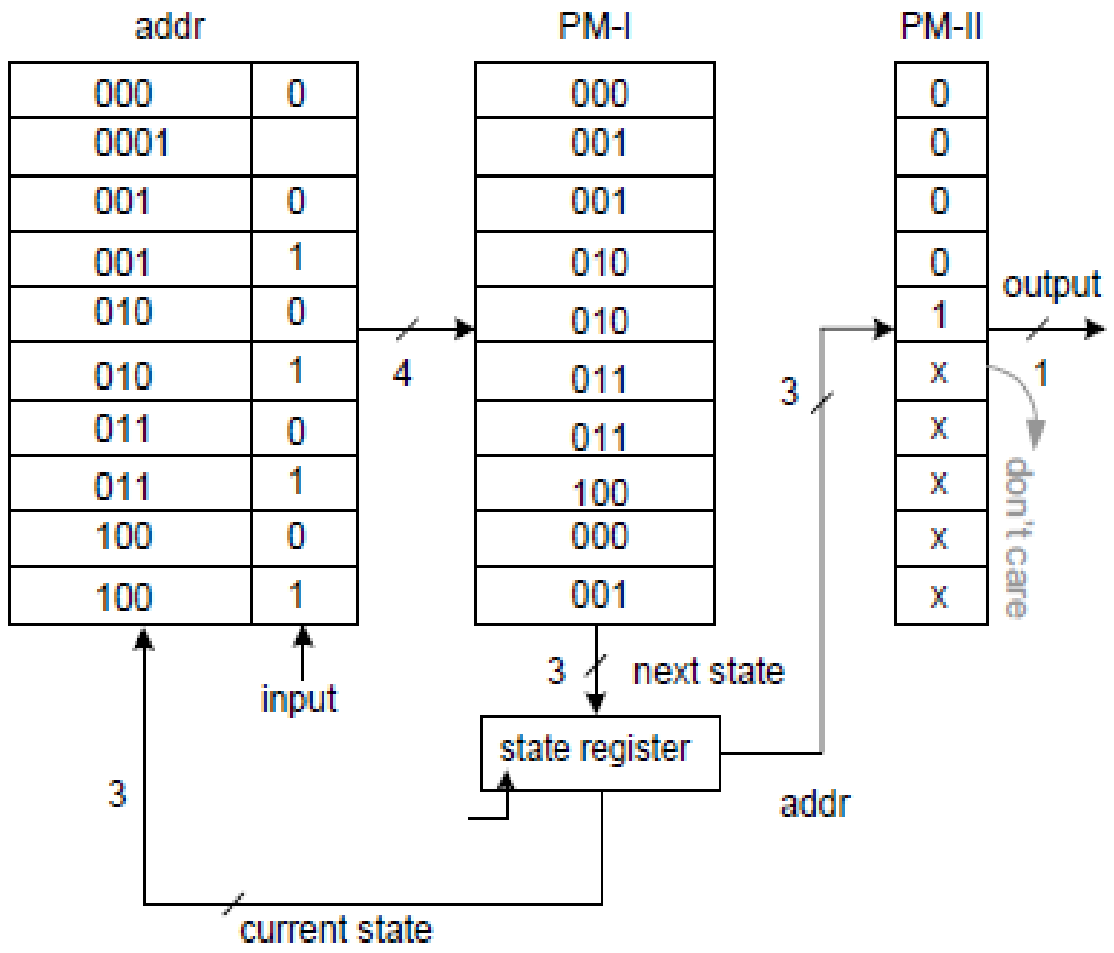
Example Mealy: Detecting four ones





Example Moore: Detecting four ones





Counter-based State Machines

- Do not depend on the external inputs.
- The sequence is stored serially in the program memory.
- The addresses can be easily generated with a counter.
- The counter thus acts as the state register
- PM only stores the control signals

Counter-based State Machines

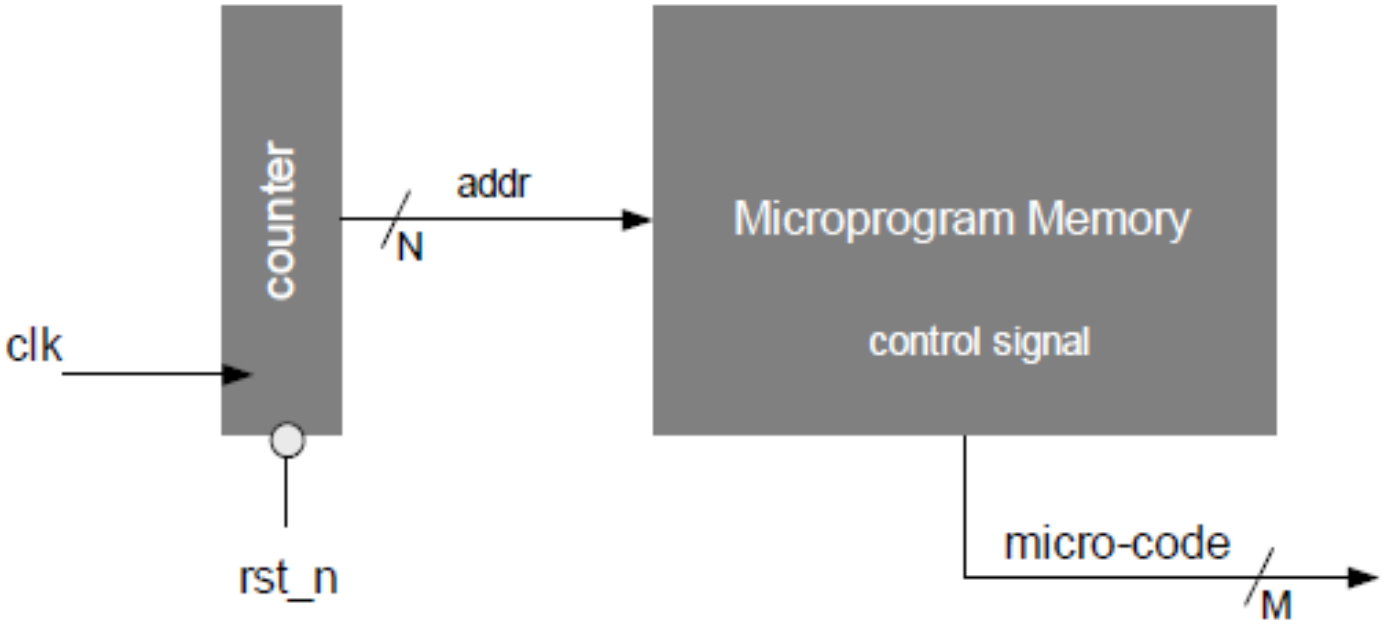


Figure 10.6 Counter based micro program state machine implementation

Loadable Counter-based State Machine

- The controller is capable of jumping to start generating control signals from a new address in the PM (Unconditional jump)

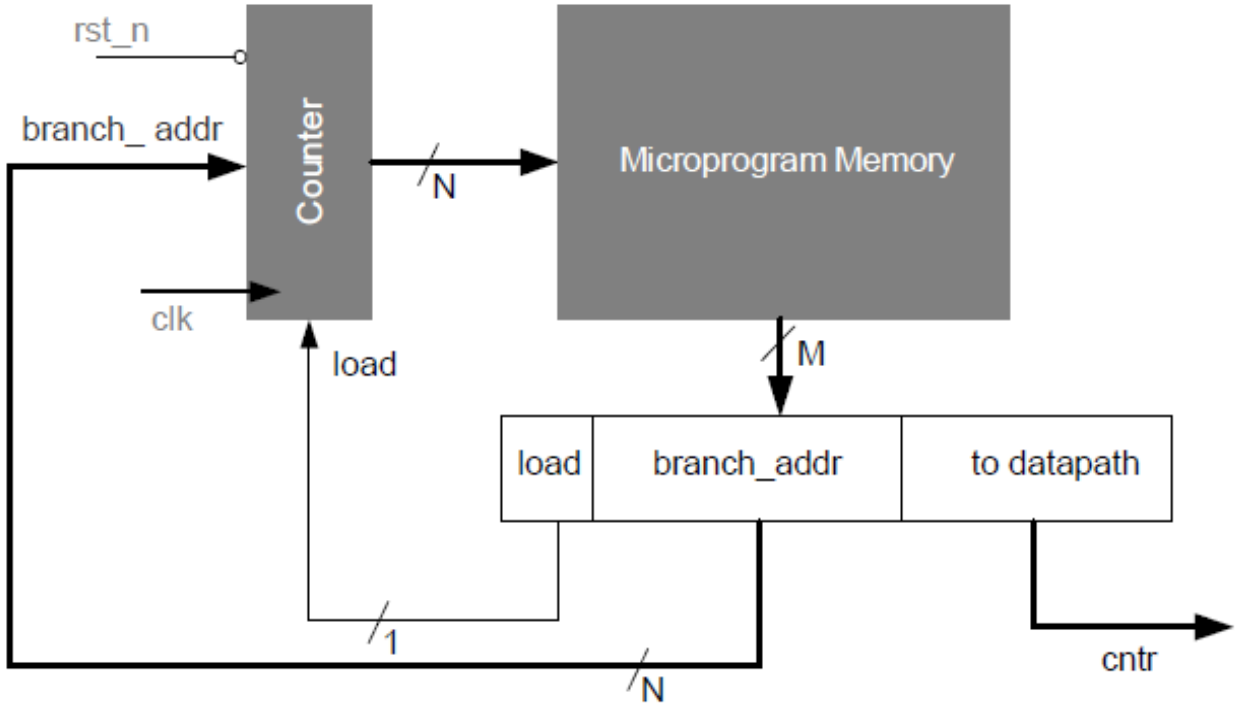
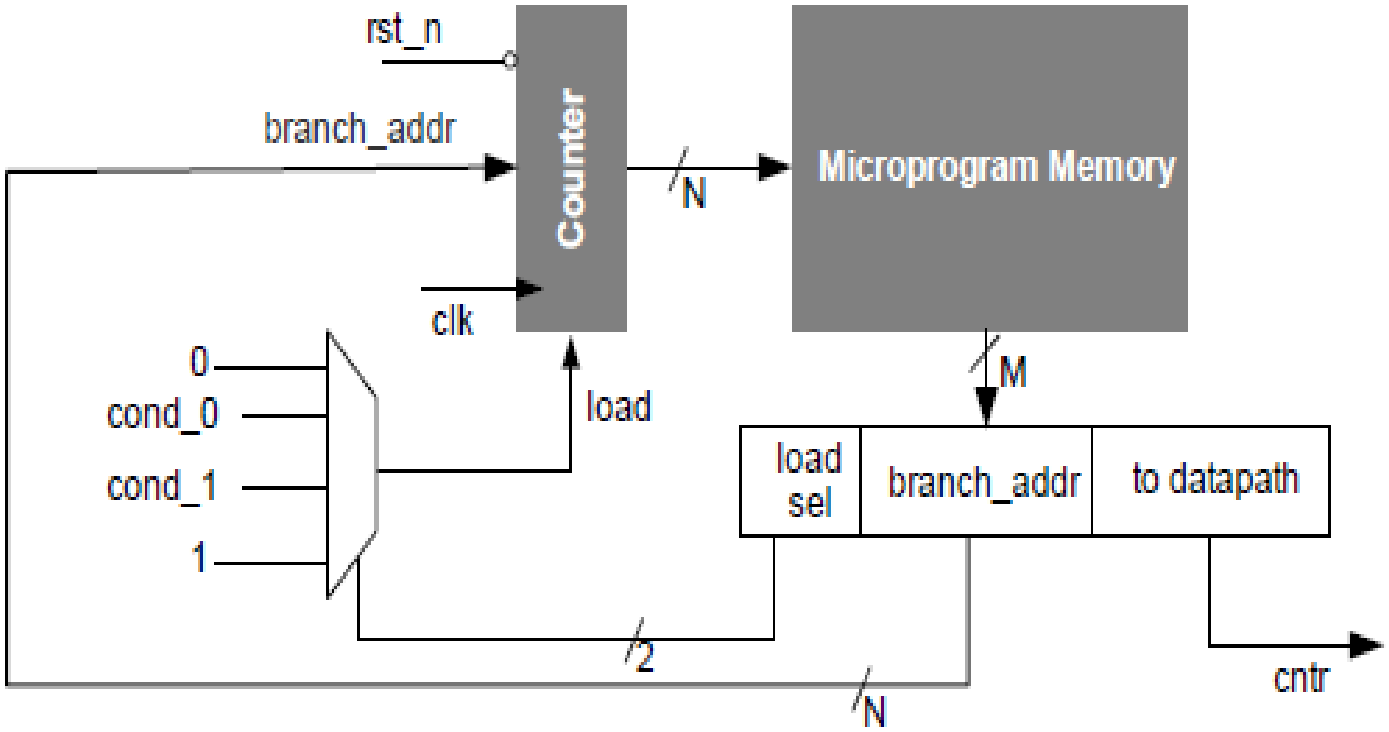


Figure 10.7 Counter based controller with unconditional branching

Counter-based FSM with Conditional Branching

- Can execute
  - if(zero flag) jump to label0
  - if(positive flag) jump to label1



Register based controller

- An adder and a register replace the counter.
- This register is called a micro program counter (PC) register.

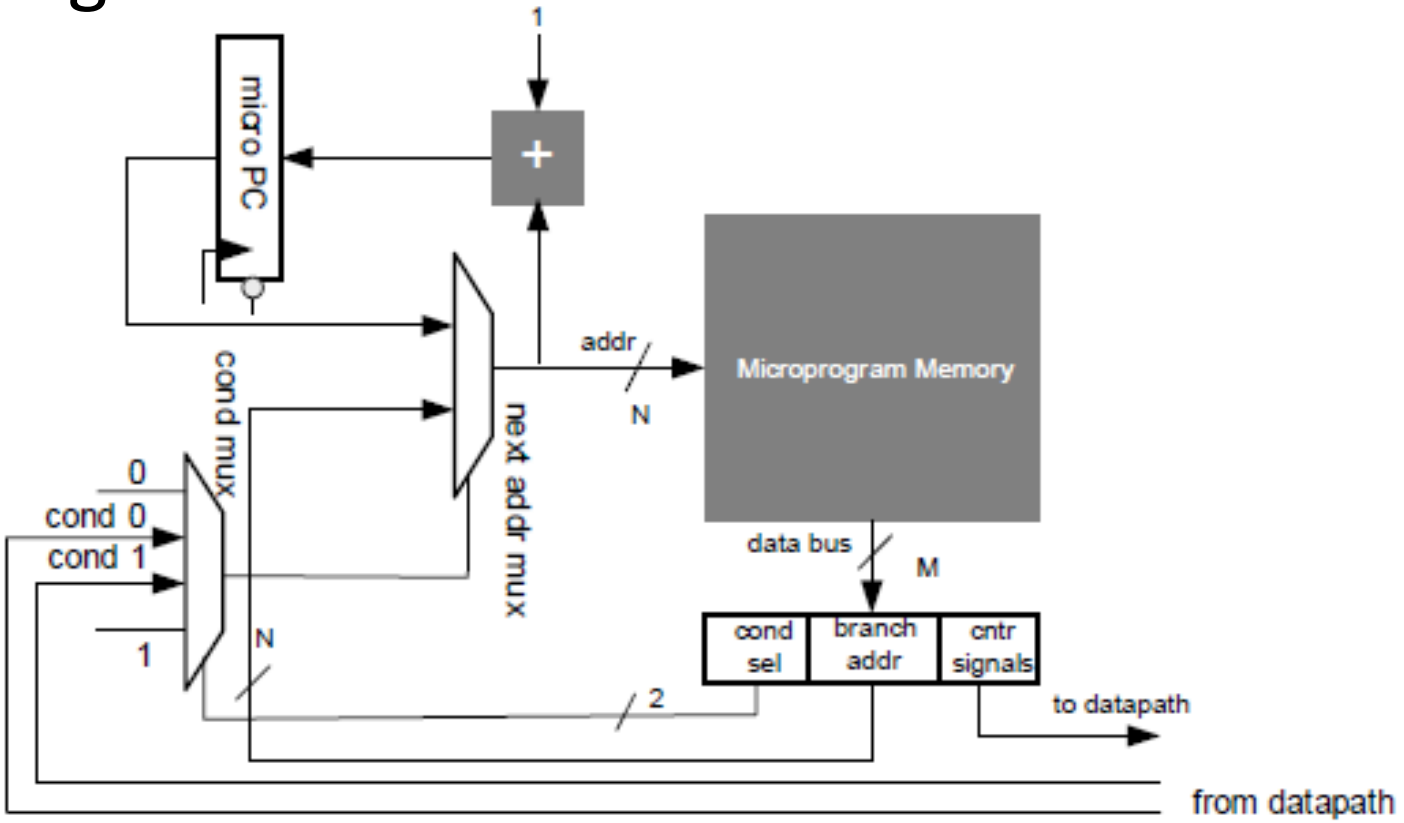
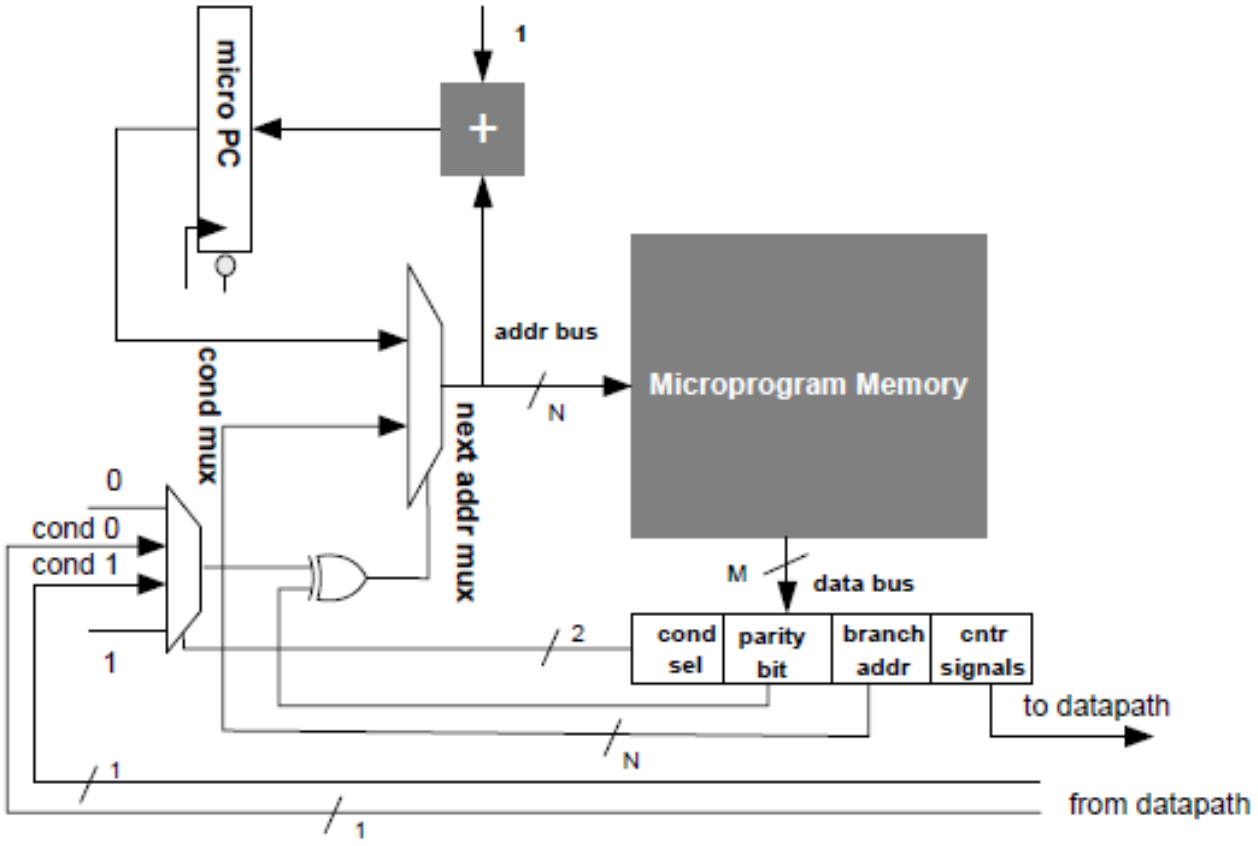


Figure 10.10 Micro PC based design

Register-based Machine with Parity Field

- The parity bit enables the controller to branch on both TRUE and FALSE states of conditional inputs
  - if (cond 0) jump to label
  - if (!cond 0) jump to label



Complete functional diagram

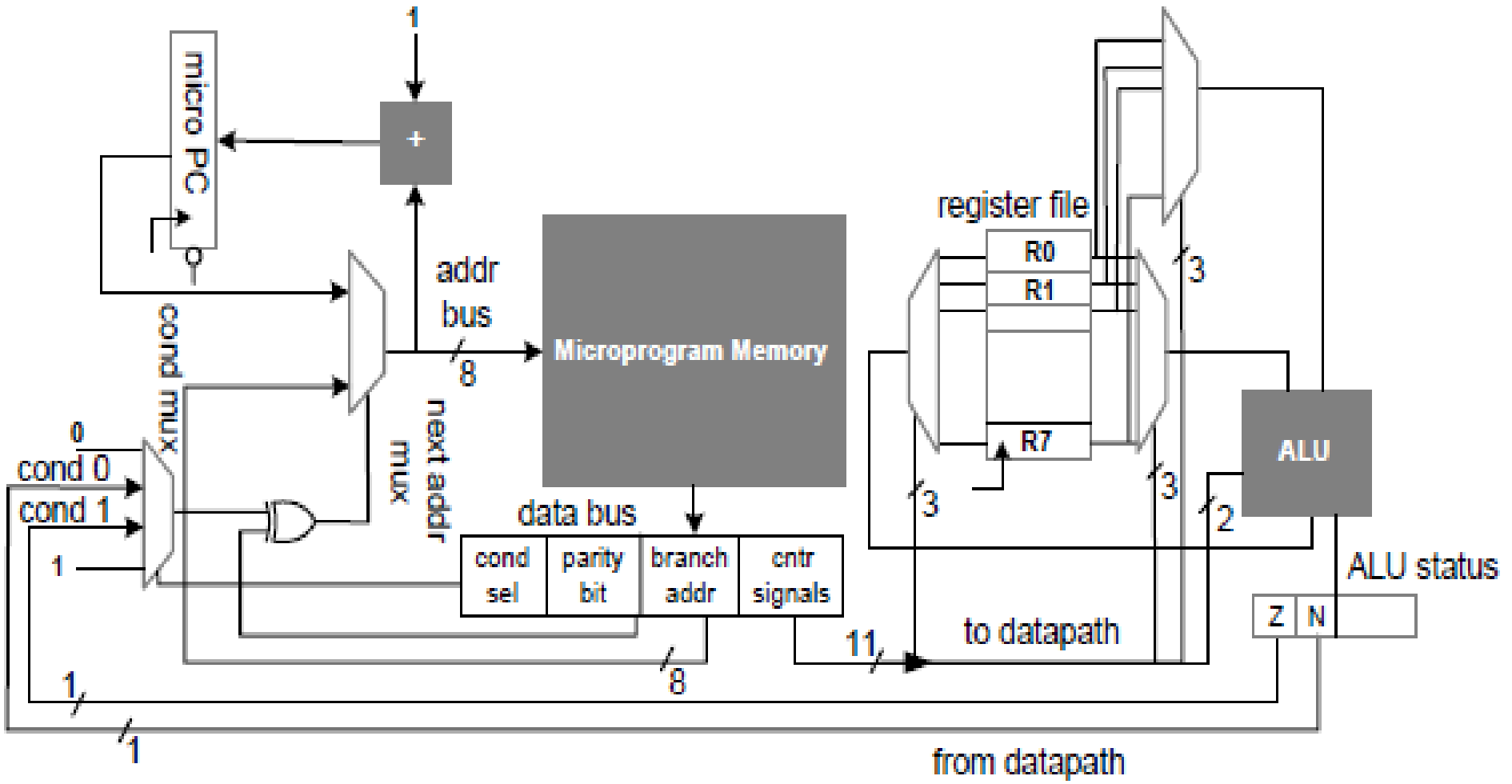


Figure 10.13 Example to illustrate complete working of a micro programmed state machine design

Instruction micro-code

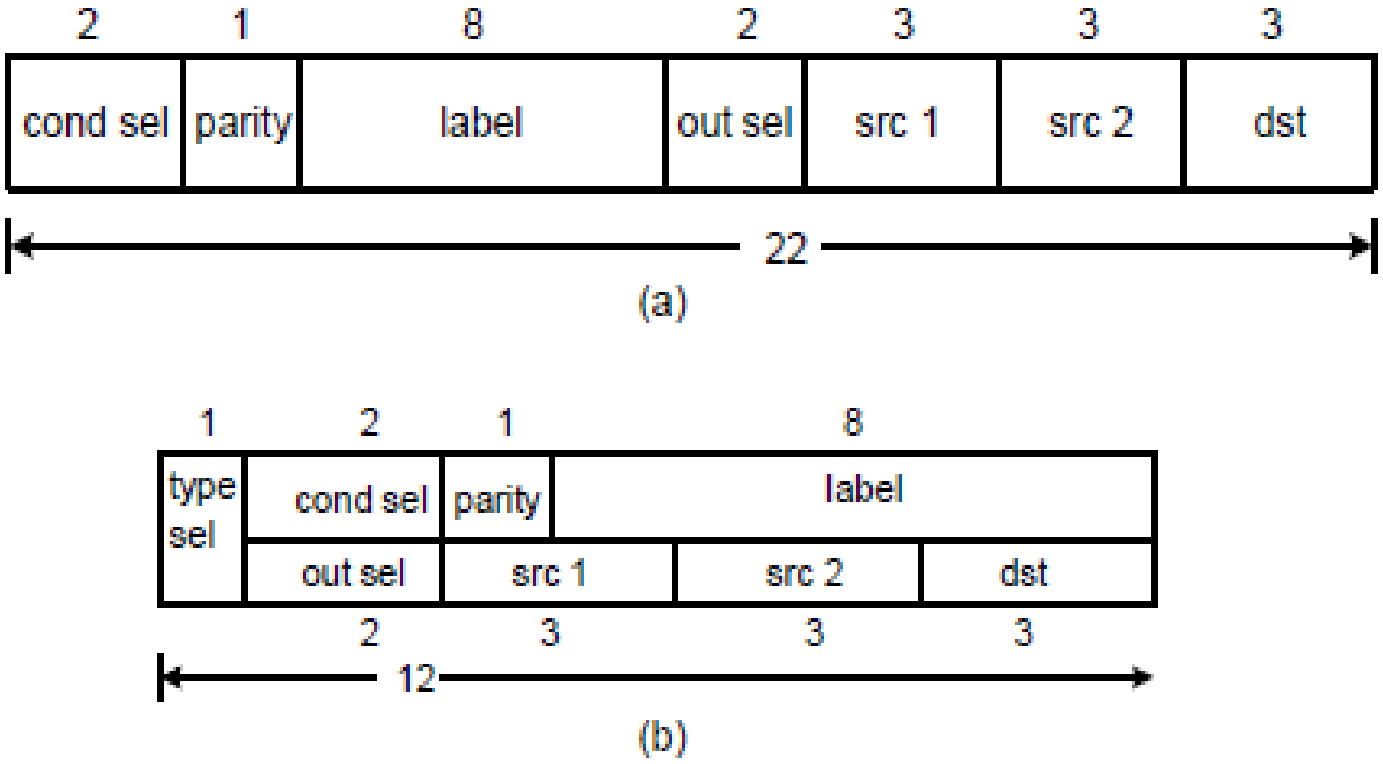


Figure 10.12 Different fields in the micro code of the programmable state machine design of the text example. (a) Non overlapping fields. (b) Overlapping fields to reduce the width of the PM

MC state machine instruction set

- The design supports a program that uses the following set of micro codes:
  - Ri Rj op Rk, for i,j,k 0, 1, 2, . . ., 7 and op is any one of the four supported ALU operations of +, -, AND, OR, an example micro code is R2 R3 + R5
  - if(N) jump label
  - if(!N) jump label
  - if(Z) jump label
  - if(!Z) jump label
  - jump label

Subroutine support

- Repeat a set of micro code instructions from different places in the micro program

```
Addr        Instr
80
81
82
83  Call 88; ->load SRA
84
85
86
87
88
89
90
91  ret; -> load PC from SRA
```

Subroutine support

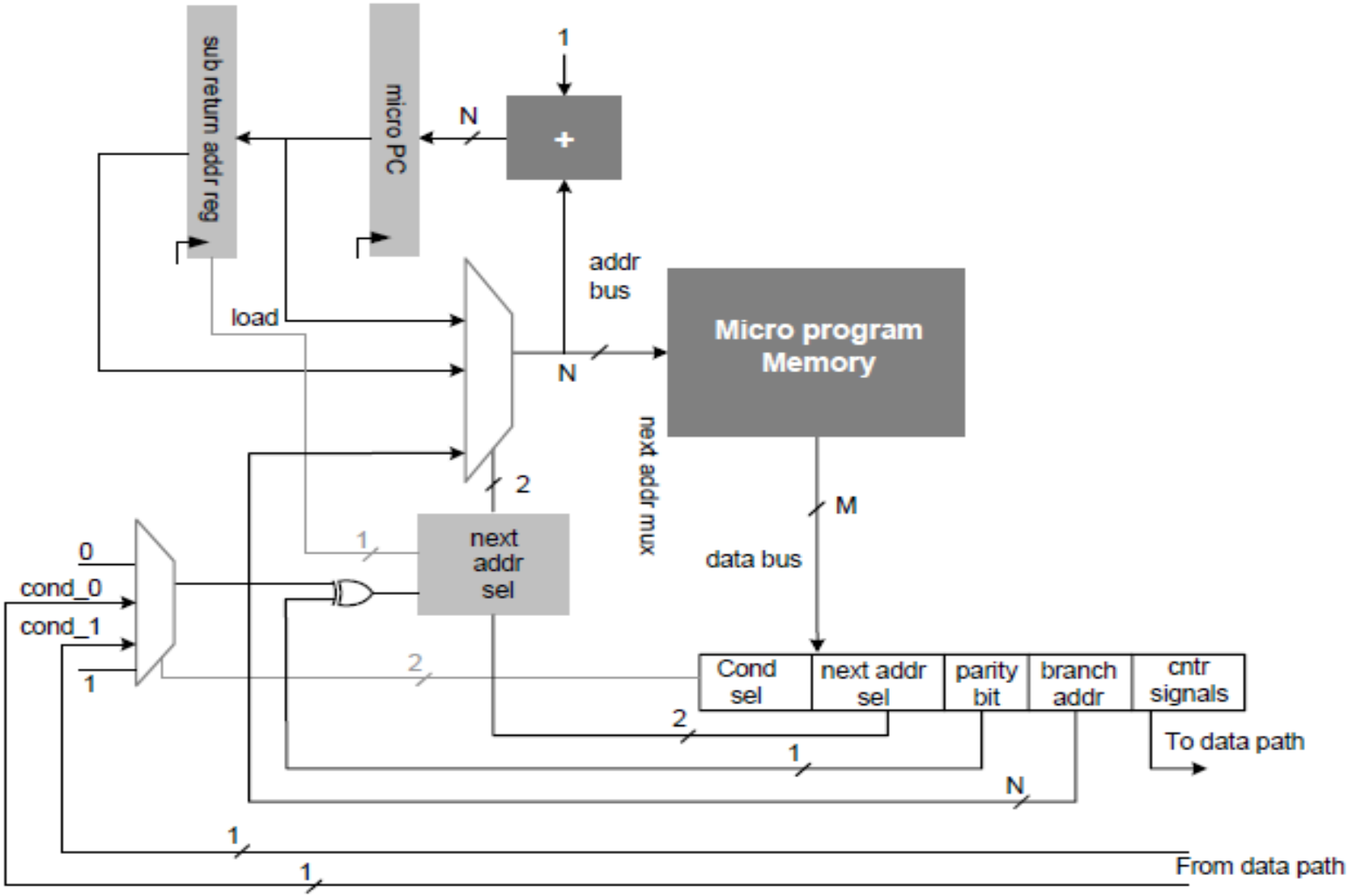


Figure 10.14 Micro programmed state machine with subroutine support

Nested subroutine support

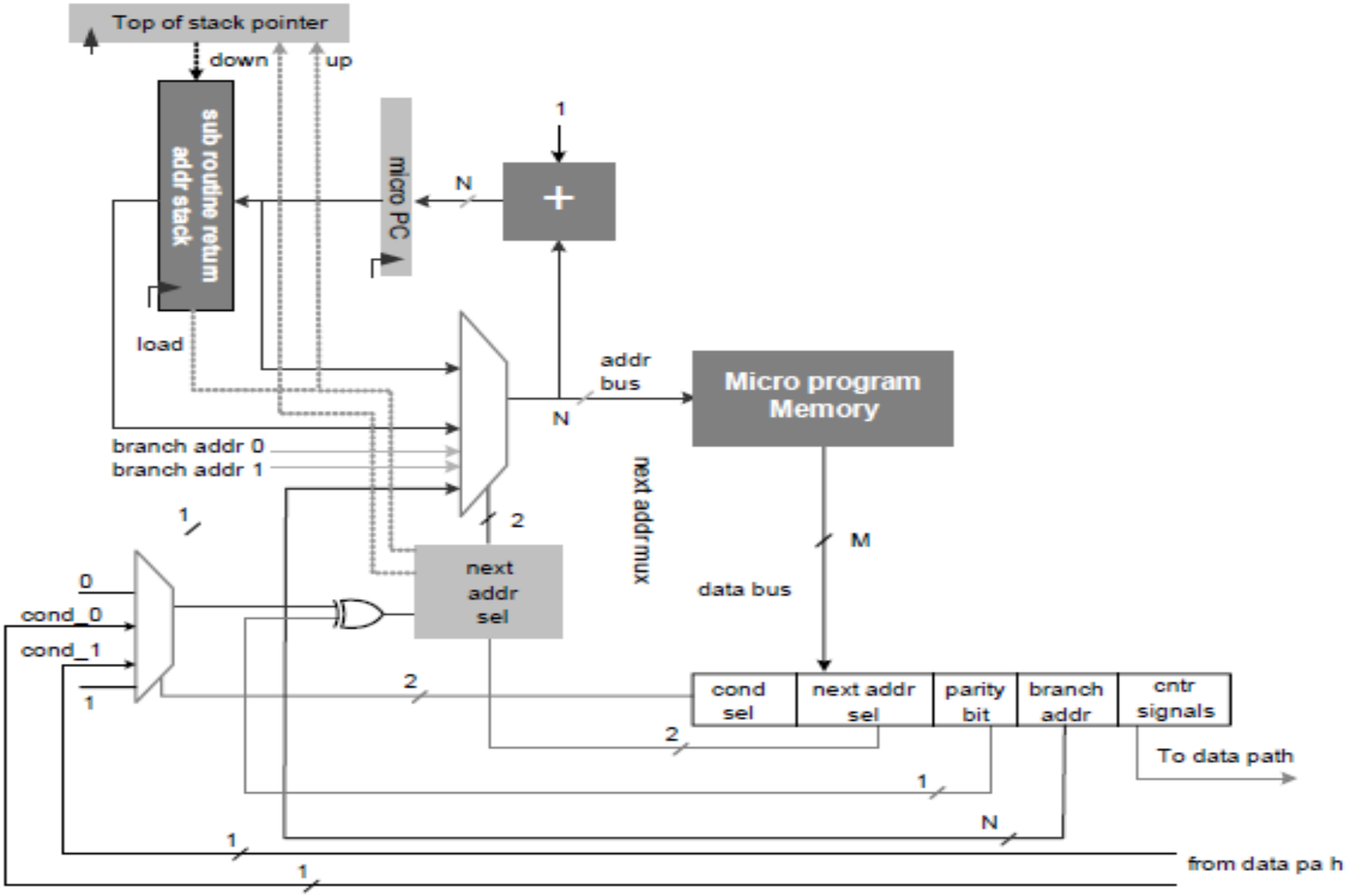


Figure 10.15 Micro program state machine with nested subroutine support

Return address stack

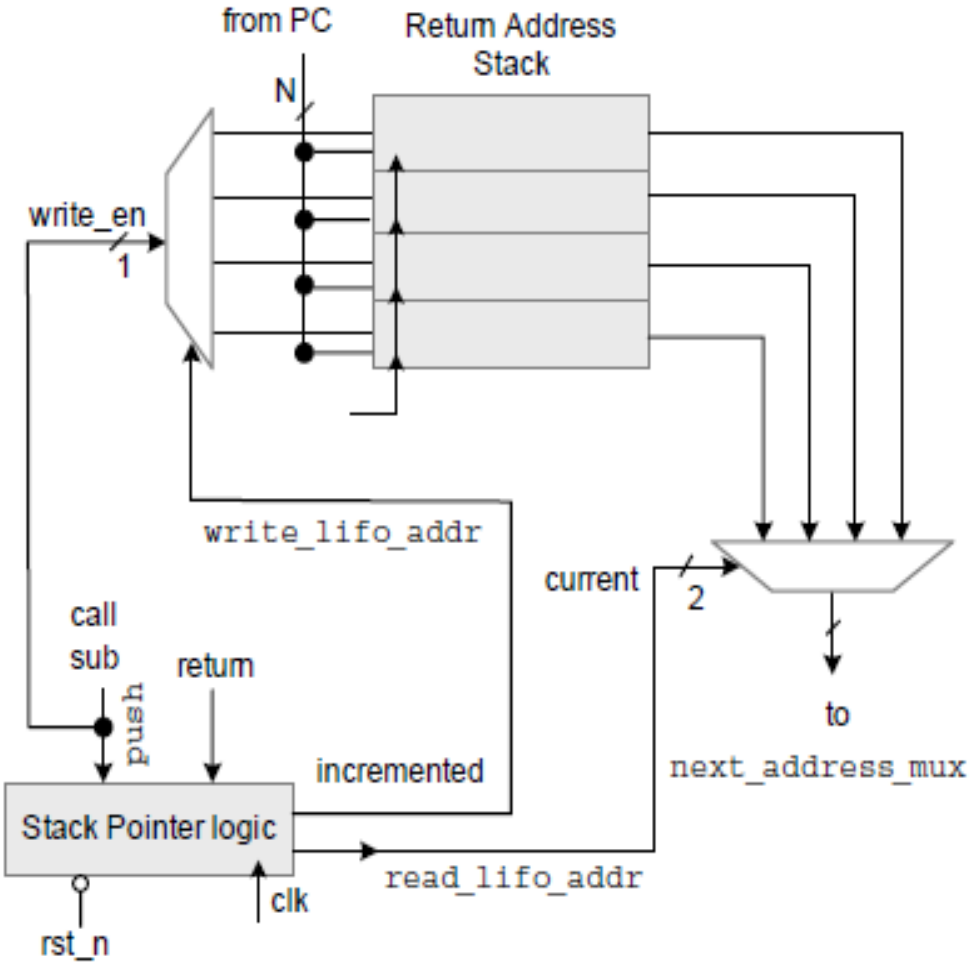


Figure 10.16 Subroutine return address stack